

```

1 import { useState, useEffect } from "react";
2
3 export const useFetch = (url) => {
4   const [dados, setDados] = useState(null);
5   // dados são renomeados para produtos em app.jsx
6   const [carregando, setCarregando] = useState(false);
7   // assegura o estado para carregando ou não dos dados
8   const [erro, setErro] = useState(null);
9   // armazena os erros encontrados durante algum dos processos do código
10  const [trigger, setTrigger] = useState(false);
11  // Força a atualização da lista, corrige erro de looping infinito ou erro de atualização por exclusão de item
12
13  // Função para configurar e executar requisições HTTP
14  const httpConfig = async (data, method) => {
15    //funcao assincronada para informar estado, dados e metodos da nossa lista
16    //data, method são passados pelo return que faz a ponte entre useFetch.jsx e app.jsx
17    setCarregando(true);
18    // altera o estado de carregando para true, afeta diretamente a nossa lisata
19    setErro(null);
20    // força o erro retornar para null antes do código voltar a rodar
21
22    try {
23      // o uso de try indica "tentativa", ou seja, tente isso
24      const config = {
25        method,
26        headers: {
27          "Content-Type": "application/json",
28        },
29        body: method !== "GET" ? JSON.stringify(data) : undefined,
30        // tudo que for diferente de GET (mesmo tipo ou não) retorna TRUE e data é transformado em objeto json
31      }; // toda essa linha simboliza a configuração generica para qualquer metodo
32
33      let requestUrl = url;
34      // salva um valor da url para realizar uma configuração
35      // Se for DELETE, adiciona o ID na URL
36      if (method === "DELETE" && data?.id) {
37        // confere se o method passando é identido
38        // usa o operador de encadeamento opcional para saber se data é TRUE (tem mesmo informações) e acessa o .id
39        requestUrl = `${url}/${data.id}`;
40        // altera o valor da requestUrl com a url atual adicionad "/" e a informação do id

```

```

41 }
42
43 const response = await fetch(requestUrl, config);
44 // faz uma busca com as seguintes informações
45 // requestUrl, informa a url, se for "delete" a url virá com o id, se for qualquer outro metodo ela vira padrão
46 // config terá sempre uma informação diferente, porém essa função não é chamada automaticamente aqui
47 if (!response.ok) {
48     // estou negando, então se relmente for false (o if se torna true) e executa o código
49     throw new Error(`Erro HTTP: ${response.status}`);
50     // throw new Error é usado dentro de try para tratar erros esperados pelo sistema e interromper o código aqui mesmo
51     // quando o erro ainda não é esperado não é uma boa prática utilizar throw new Error
52 }
53
54 setTrigger((prev) => !prev);
55 // Força a atualização dos dados após a requisição
56 // Um gatilho que atera o seu statu apenas para que o hook a seguir seja obrigado a renderizar novamente
57
58 return response.status !== 204 ? await response.json() : null;
59 // Retorna o JSON apenas se houver conteúdo
60 // verificifica o status de response
61 // se for diferente de 204 (no content) retorna um objeto json
62 // se for 204 ele automaticamente retorna null
63 } catch (error) {
64     // parte do try que caputra o erro, porém inesperado
65     setErro(error.message);
66     // muda status para armazenar a mensagem do erro pego em catch
67     throw error;
68     // finaliza o código lançando o erro de volta
69 } finally {
70     // parte final da estrutura try
71     // SEMPRE executa, mesmo que throw new Error interrompa o código
72     // é uma boa pratica inseirir, já que executa o encerramento do estado de carregamento
73     setCarregando(false);
74     // altera o estado de carregamento
75 }
76 };
77
78 // Efeito para buscar os dados
79 useEffect(() => {
80     const fetchData = async () => {
81         // função assincrona para fazer a solicitação dos dados

```

```

82   setCarregando(true);
83   // mesmo que acima o carregamento terminou, ali era apenas o "envio das configurações"
84   setErro(null);
85   // limpamos o estado de erro para realizar uma nova etapa do código e verificar se há erros nessa parte
86
87   try {
88     // tente
89     const response = await fetch(url);
90     // reponse vai aguardar (await) a busca (fetch) da base (url)
91     if (!response.ok) {
92       // estado de false, se for false executa a proxima linha
93       // se reponse.ok for true o código contia a proxima etapa
94       throw new Error(`Erro HTTP: ${response.status}`);
95       // interrompe o fluxo do código e exibe a mensagem dde erro
96     }
97
98     const json = await response.json();
99     // cria o objeto json
100    // espera (await) a resposta (response) ser tranformada em um objeto json (.json())
101    setDados(json);
102    // altera o estado de dados com o novo objeto json
103  } catch (error) {
104    setErro(error.message);
105    // se der algum erro inesperado, faz a captura do erro e emite ele na tela
106  } finally {
107    setCarregando(false);
108    // mesmo que de erro finaliza, e altra o estado de carregamento
109  }
110 };
111
112 fetchData();
113 // reiniciamos nosso código
114 // basicamente o código é "carregado", mas precisamos chamar ele fora de sua estrutura para executar
115 }, [url, trigger]);
116 // aqui estao as dependencias:
117 // url - se ela mudar o useEffect é executado novamente
118 // trigger - sempre vai ser alterado (pela função) httpConfig forçando o useEffect ser executado novamente
119 return { dados, httpConfig, carregando, erro };
120 // aqui estamos "exportando" para que possamos acessar em app.jsx
121 };

```